Measuring and Mitigating Memory Drift in Long-Term LLM Agents: Render-from-Raw Memory with NLI-Triggered Refresh

M.E. Research Proposal – Second-Brain Memory for LLM Agents (4 November 2026)

Selvin PaulRaj K
M.E – Computer Science Engineering,
Kings Engineering College,
Chennai, Tamil Nadu,
210825405017@kingsedu.ac.in

***Abstract* - Long-term memory lets a large language model (LLM) agent remember a user across weeks of conversations. Many widely used memory systems keep the store small by asking an LLM to rewrite it after every session, so each version is derived from the previous one rather than from what the user said. We hypothesise that such rewrite-in-place memory loses information as cycles accumulate, much as errors propagate in a predictive video codec until a new reference frame is coded. Prior work reports degradation under repeated updates, but, to our knowledge, none measures question-answering accuracy on conversational memory against the number of rewrite cycles. We propose three contributions: (1) DriftBench, a protocol that applies $k = 0, 1, 2, 4, 8$ consolidation cycles to LoCoMo, Memora and LongMemEval memories at a matched token budget, with a content-controlled arm that separates rewrite damage from added data; (2) a testable information-theoretic prediction of the drift curve, compared against the alternative that drift settles to an equilibrium; and (3) Render-from-Raw memory, which keeps an immutable raw log and rebuilds a memory from it when a natural language inference (NLI) and fact-coverage check finds it contradicted or incomplete. A Tanglish (romanised Tamil–English) extension examines read-time normalisation. Experiments are designed for free-tier API models and a CPU. No results are reported yet.**

***Keywords* —** *LLM Agents, Long-Term Memory, Memory Drift, Consolidation, Event Sourcing, Provenance, Retrieval-Augmented Generation (RAG), BM25S, Natural Language Inference, Tanglish.*

1. INTRODUCTION

An LLM agent that helps one person for months needs a memory outside its context window: the user's preferences, plans, people and past decisions. Systems such as MemGPT [1], Mem0 [2] and A-MEM [3] provide this "second brain" by storing notes and retrieving the relevant ones at answer time, in the manner of Retrieval-Augmented Generation (RAG) [4].

To keep the store small, these systems *rewrite* it. After a session an LLM decides what to add, update, merge or delete, and the result replaces what was there. The next session's rewrite starts from that result, not from the original conversation. If each rewrite drops or distorts a little, the errors compound. Zhang et al. [5] reported that LLM-consolidated memories first help and then degrade, sometimes below a no-memory baseline, while simply keeping the raw episodes stays competitive. Zerhoudi et al. [6] reported a "compaction cliff": on 20 production agent configurations, a widely used compaction prompt kept 53% of safety rules after one round and 10% after five. Kwon [7] went further and argued that a lossy memory can be worse than an empty one.

Video coding met the same problem decades ago. A P-frame is predicted from an earlier frame, so a prediction error propagates down the chain until the next intra-coded I-frame is rebuilt from the source picture [8]. Table 1 shows the analogy that motivates this work: rewrite-in-place memory is a chain of predictions, while a memory rebuilt from the raw log behaves like a periodic I-frame.

Raw logs are not new: several 2026 systems keep one [9], [10], [11], [12]. That raw evidence should not be overwritten is therefore already argued [5]; we do not claim it. Colaco and Lahjouji [13] note that the repeated compaction agents actually perform is almost never measured, and Dongre et al. [14] find that multi-turn drift settles into bounded equilibria rather than runaway decay. Whether memory drift under repeated rewriting decays or plateaus is open. What is missing is a *measurement* of how much rewrite-in-place costs on conversational memory as cycles accumulate, with rewrite damage separated from the arrival of new data, and a cheap way to decide *when* a rendered memory must be rebuilt from the raw text. This work makes three contri-

TABLE 1 – VIDEO-CODING CONCEPTS AND THEIR MEMORY ANALOGUES

Video coding	**Second-brain memory (this work)**
Source frames	Immutable raw log of every turn
P-frame chain	Rewrite-in-place: each summary built from the last
I-frame refresh	Render-from-raw: rebuild a memory from its source
Drift / error propagation	Facts lost or distorted over rewrite cycles
Edit decision list (non-linear editing)	Corrections stored as operations, replayed on every render
Colour grade / LUT	Versioned read-time normalisation (script, names, units)
Container seek index	BM25S and time index for exact relink to raw

butions: (1) DriftBench, a released harness with a fixed-content arm and a streaming arm that measures question-answering accuracy against the number of consolidation cycles at a matched token budget, on LoCoMo [15], Memora [16] and long LongMemEval [17] histories, designed so that a plateau is as reportable as a decay; (2) a testable prediction of the curve's shape, motivated by the strong data-processing inequality, with the per-cycle fact-survival rate estimated from data; and (3) Render-from-Raw memory with an NLI-triggered refresh: a staleness detector that catches both contradictions and omissions, scored against oracle staleness labels. A layer-aware router and a Tanglish read-time normalisation layer are designed (Section 3) but are extensions, not contributions; they are kept only if time allows.

2. RELATED WORK

A. Long-Term Memory for LLM Agents

MemGPT [1] pages memory between context and storage; Mem0 [2] extracts facts and applies ADD, UPDATE and DELETE operations; A-MEM [3] links notes Zettelkasten-style and lets new notes rewrite old ones. TiMem [18] consolidates segments into session, day, week and profile levels, and LightMem [19] moves consolidation to an offline "sleep-time" pass. All of them store the output of an LLM rewrite as the memory itself.

B. Raw Logs and Event-Sourced Memory

Srinivasan's Decision Memory [9] keeps an append-only event log and builds one task-conditioned projection at decision time, and shows that this beats summarisation memory as the budget tightens. User as Code [10] never discards a fact and periodically checkpoints its log into typed code; PROJECTMEM [12] regenerates a summary by a deterministic fold over the log after every write; MemFit [11] stores turns verbatim and uses summaries only as an index. All-Mem [20] performs non-destructive consolidation with Split, Merge and Update edits while preserving immutable evidence, and expands from active memories to archived evidence at query time, close to the relink path proposed here. TrajWiki [21] records ADD, REVISE and DEPRECATE operations over fixed snapshots, close to an edit list. APEX-MEM [22] stores an append-only property graph and resolves conflicts at query time, and AnchorMem [23] keeps the original context immutable and criticises the retrieve-and-rewrite loop. Version control and rollback of memory appear in ChronoMem [24], Shepherd [25] and DeltaMem [26]. A production design, CortexDB [27], computes every derived view as a function of the events and a derivation version. The architecture proposed here is therefore *not* claimed as new.

C. Drift Under Repeated Rewriting

Zhang et al. [5] report utility rising then falling as an LLM keeps consolidating agent trajectories on ARC-AGI, not on conversational question answering. The Compaction Cliff [6] tracks safety-rule retention, not facts about a user, over five rounds, and Reclaim [7] shows compounding through memory loops. Dongre et al. [14] model multi-turn context drift as a bounded stochastic process and observe stable, noise-limited equilibria; their setting is the model's outputs across turns, not a stored memory, but it is the competing hypothesis for H1. Lam et al. [28] propose periodic replay against drift as a governance mechanism without an evaluation. Colaco and Lahjouji [13] give a rate–distortion view of compaction with a lower bound and a benchmark proposal, and flag repeated compaction as unmeasured; this is the closest scoop risk. None of these plots question-answering accuracy against the number of cycles on conversational long-term memory, or compares rebuilding from raw with rewriting in place at the same budget. Song et al. [29] run retrieval-driven reconsolidation on LoCoMo and LongMemEval and find that it *improves* accuracy, and a streaming LongMemEval-M study [30] finds a consolidation pipeline matching raw retrieval (70.1% vs 71.2%) at a 200K-token budget, so consolidation is not always harmful. Another counter-result matters: an IVA 2026 study [31] found that slot overwriting beat append-only memory on recall, so the benefit of raw logs is not automatic.

D. Retrieval for Memory: Raw versus Summary

Zep [32] already combines dense vectors, BM25 and a temporal knowledge graph. Mnemis [33] lets an LLM navigate a hierarchy without embeddings, and SelRoute [34] and BudgetMem [35] route queries by type or budget. The closest work to the router proposed here is JustMem [36], which chooses per query between compact atomic memory and replaying the original conversation, and TierMem [37], which answers from summaries and escalates to an immutable raw tier when the summary is insufficient. Memora [16] targets consolidation and knowledge updates over weeks to months of conversation, with the FAMA metric that penalises use of obsolete memories; it is the benchmark most likely to stress rewriting. RENDER [38] controls the reader-facing evidence in memory evaluation and is used here as an evaluation control. BM25S [39] makes exact lexical search over a growing raw log cheap on a CPU.

E. Provenance, Security and Deletion

Agent Zero Memory [40] attaches provenance to long-term memories. Xu et al. [41] describe *provenance laundering*, where an LLM rewrite turns an untrusted note into what looks like user history. MemTrust [42] proposes a zero-trust memory architecture and MemCut [43] shows that deleted secrets leave traces in agent memory, which an append-only design must address. TOKI [44] resolves contradictions with a bitemporal operator algebra.

F. Research Gap

Raw logs, edit operations, read-time projection and raw-versus-summary routing each exist. What no surveyed work provides is (a) a controlled, budget-matched curve of question-answering accuracy against the number of rewrite cycles on conversational long-term memory that separates rewrite damage from new data and distinguishes decay from equilibrium; (b) a measured estimate of how fast facts survive each rewrite; (c) a refresh policy triggered by detected contradiction *or omission* relative to the raw source, with the detector's own error rates reported; and (d) any test of read-time normalisation when the same personal fact arrives in Tamil script, romanised Tamil and English. This work targets (a)–(c) as its core and (d) as an extension, and and claims nothing about the architecture itself.

3. METHODOLOGY

A. Existing versus Proposed System

Table 3 contrasts the rewrite-in-place design used by many widely used memory systems with the proposed Render-from-Raw design.

B. Proposed System Architecture

In a predictive video codec, a P-frame is coded from an earlier decoded frame, so errors accumulate until an I-frame is rebuilt from the source; we apply the same remedy to memory. Two terms are used throughout. The *grade layer* is a versioned normalisation policy (names, units, dates, script) applied when memory is read, by analogy with colour grading applied to footage without altering it. A *render receipt* is the record returned with every answer naming the raw spans, edit-list version and grade version it was built from. Figure 1 shows the design. Every user or agent turn is appended to the raw log L and never edited. A correction ("I moved to Chennai") is appended to an edit list E as an operation. A rendered memory R is always produced from the source, $R = g_v(\text{apply}(E, L))$, where g_v is grade profile version v; it is never produced from a previous R.

C. Components

Raw log. Turns are stored verbatim in segments, each encrypted under its own key so that a deletion request can be honoured by destroying the key and re-rendering (crypto-shredding), a response to the leakage shown by MemCut [43].

Edit list. Corrections are operations with a timestamp and a pointer to the raw turn that caused them. Because every render replays E, a correction cannot be lost by a later summary, which is the failure mode a rewrite chain allows.

Grade layer. g_v is a deterministic or LLM-prompted normalisation policy (canonical names, units, dates and, in the Tanglish extension, script). It is versioned, so changing it re-renders memory without touching L, and an old answer can be replayed exactly with its render receipt.

Staleness detector and refresh. Most drift is omission: a summary that silently drops a fact is not contradicted by the raw span. The detector therefore runs two checks. (i) *Contradiction*: an NLI model compares each rendered memory with the raw span it cites. (ii) *Coverage*: at write time each raw segment is split once into atomic facts, which are cached; a fact counts as *missing* when no rendered memory entails it above a threshold τ. Raw spans longer than the NLI input limit are split into overlapping 400-token windows and the maximum entailment score is kept. A memory with a contradiction, or a segment with missing facts, is marked stale and only those are rebuilt, so refresh cost follows drift rather than memory size. The detector is scored for precision and recall against oracle staleness labels derived from each benchmark's gold evidence turns. The closest prior work is AMA [45].

Layer-aware router (extension). A rule-based router, later replaced by a small classifier, sends exact questions (names, numbers, dates, quoted text) to BM25S over the raw log, gist questions to dense retrieval over rendered memories, relational questions to the knowledge graph, and time or topic questions ("what did I plan in March?") to a reasoning tree. When the top rendered memory has low confidence or an NLI flag, the router relinks to the raw span. The knowledge graph and the tree are kept in the main paper only if an ablation shows a gain of at least three points.

D. Drift Prediction

Let $H(X)$ denote the entropy of the source sessions and $I(X; M_k)$ the mutual information between them and the memory after k consolidation cycles. In the *fixed-content* arm, $M_k = f(M_{k-1})$ with $M_0 = X$. In the *streaming* arm, $M_k = f(M_{k-1}, S_k)$, where S_k is the session added at cycle k.

Proposition 1. Assume (A1) for a fact from session S_1, later sessions $S_2, \ldots, S_k$ are independent of S_1, so $S_1 \to M_1 \to \cdots \to M_k$ is a Markov chain; and (A2) each consolidation step contracts information by a strong data-processing coefficient $\eta < 1$ [46]. Then

$$I(S_1; M_k) \le \eta^{k-1} I(S_1; M_1) \le \eta^{k-1} H(S_1). \qquad (1)$$

Neither assumption is known to hold for LLM rewriters (A1 fails when a later session repeats an earlier fact), so Eq. (1) is used as a prediction of the curve's shape, not as a guarantee. For a render-from-raw memory, $I(X; R_k)$ does not depend on k by construction, so that comparison is not the claim. The

TABLE 2 – CLOSEST PRIOR WORK AND THE GAP RELATIVE TO THIS WORK

Work	What it does	Overlap with this work	Gap relative to this work
Decision Memory [9]	Append-only log, one projection at decision time	Raw log, read-time rendering	Enterprise decisions; no drift curve over cycles
JustMem [36]	Per-query choice: compact memory or replay original	Layer-aware routing	No refresh policy; no drift measurement
TierMem [37]	Summary first, escalate to immutable raw tier	Relink to raw	Two tiers; refresh only on demand
User as Code [10]; PROJECT-MEM [12]	Log never discards; periodic or per-write rebuild	Render-from-raw	Rule-based fold, not LLM rendering; no QA drift curve
Faulty memories [5]; Compaction Cliff [6]	Utility rises then falls (ARC-AGI); safety rules kept 53% → 10% in 5 rounds	Drift over cycles	Not conversational QA; no budget-matched raw baseline
TrajWiki [21]	ADD / REVISE / DEPRECATE over snapshots	Edit list	No read-time grade layer
Zep [32]	Vector + BM25 + temporal KG	Hybrid retrieval	No raw-versus-rendered choice
REALM [29]	Reconsolidation over rounds on LoCoMo, LongMemEval	Cycle protocol	Reconsolidation helps; no content-controlled drift curve
All-Mem [20]	Non-destructive edits; links to immutable evidence	Raw anchoring, relink	No drift measurement; no omission detector
Drift No More? [14]	Drift as bounded equilibrium	Shape of the curve	Output drift, not stored memory
IVA 2026 [31]	Overwrite beats append-only on recall	Counter-evidence	Must be reproduced and explained

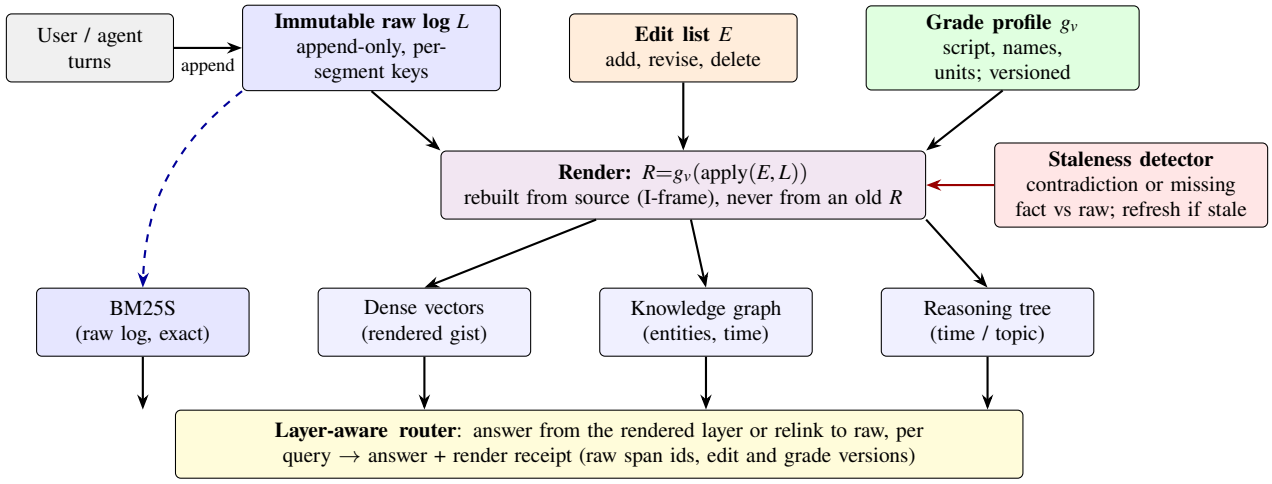


Figure 1: Render-from-Raw memory. Solid arrows: rendering path; dashed: exact relink path from the raw log to BM25S.

TABLE 3 – EXISTING VERSUS PROPOSED MEMORY DESIGN

Aspect	Rewrite-in-place (Mem0-style)	Render-from-Raw (proposed)
Stored truth	Latest LLM rewrite	Immutable raw log
Corrections	Overwrite the note	Edit list, replayed on render
Normalisation	Fixed at write time	Versioned grade, applied at read time
Refresh	Never, or every session	When the detector finds a contradiction or omission
Retrieval	Dense over notes	Router: rendered layer or raw (BM25S)
Provenance	Lost after merge	Render receipt: raw span ids + versions
Deletion	Delete the note	Per-segment key shredding, then re-render

claim is empirical: η for an LLM rewriter is unknown and probably close to 1, so we estimate a per-cycle fact-survival rate $\hat{\eta}$ by fitting retention $\approx \hat{\eta}^k$ on the fixed-content arm, reports it per writer model, and tests whether $\hat{\eta}$ predicts the streaming curve. A curve that levels off at a non-zero asymptote supports the equilibrium view [14] instead; either outcome is reported.

E. Tanglish Extension (Planned)

Personal memory for a Tamil-speaking user mixes scripts: the same fact may arrive as கட்டணம் செலுத்த கடைசி தேதி, *fees kattura last date* or "fee payment deadline". A rewrite chain may store all three as separate, conflicting notes. The grade layer can instead normalise script at read time using a Tanglish-to-Tamil transliterator from our earlier M.E. thesis work and IndicXlit [47], and the raw log keeps the original form. Seed facts can be drawn from the IndicTalk corpus [48].

4. EXPERIMENTAL DESIGN

A. Data and Protocol

DriftBench has two arms. The *fixed-content* arm builds memory from a fixed set of sessions and re-consolidates the same content $k \in \{0, 1, 2, 4, 8\}$ times; only rewriting changes, so any loss is pure rewrite damage, the headline claim. The *streaming* arm presents the next block of sessions at each cycle, as a deployed agent would. LoCoMo [15] conversations are short enough that full context may beat every memory, and many of its questions may need only recent sessions, so a flat curve there could mean the benchmark does not stress consolidation. The main results therefore use Memora [16], long LongMemEval [17] histories and a synthetic stream that exceeds the reader's context window. Each question is tagged by the age of its gold evidence (sessions since it was said), and accuracy is reported per age band, so a flat average cannot hide decay on old facts. LoCoMo answer keys used in the pilot are spot-checked by hand. The reader and its evidence are held fixed with the RENDER controls [38].

B. Systems Compared

(i) Rewrite-in-place in the style of Mem0 [2]; (ii) full-context and raw-only BM25S baselines, which are flat in k by construction; (iii) TierMem-style escalation [37] and JustMem-style replay [36], reimplemented where code is unavailable; (iv) Render-from-Raw with a scheduled refresh; and (v) Render-from-Raw with detector-triggered refresh. All systems receive the same token budget per query and per cycle, and results are reported as accuracy against tokens, so a flat baseline wins only if it is also affordable.

C. Metrics and Hypotheses

Metrics are answer accuracy, fact retention, refresh and query tokens, CPU latency, and detector precision and recall. The judge is a stronger model (Llama 3.3 70B) than the memory writer, with exact match on short answers, validated against 100 hand-labelled answers. The memory writer is varied (8B and 70B) on a 100-question subset, so drift is not merely a weak rewriter. A pilot of 250 questions sets the sample size: if the minimum detectable paired difference exceeds 5 points, the set grows to about 500. Paired bootstrap and McNemar tests are used throughout.

H1. In the fixed-content arm, rewrite-in-place accuracy on old-evidence questions falls by at least 5 points between $k = 1$ and $k = 8$, approximately log-linearly in k. **H1′ (competing).** It plateaus at a bounded equilibrium [14]; the two are compared by model fit (exponential versus exponential-to-asymptote). **H2.** $\hat{\eta}$ fitted on the fixed arm predicts streaming-arm accuracy within 3 points at every k. **H3.** On the accuracy-per-token frontier, Render-from-Raw with detector-triggered refresh dominates rewrite-in-place and matches scheduled refresh (within 2 points) with at least 30% fewer refresh tokens, and the detector reaches recall of at least 0.8 on oracle staleness labels. **H4 (extension).** The router beats always-rendered and always-raw retrieval. **H5 (extension).** Read-time script normalisation beats write-time normalisation on cross-script recall.

D. Environment

Llama 3.1 8B [49] and Llama 3.3 70B on the Groq API at temperature 0; BM25S and the NLI model on a CPU. Long histories raise the token cost well above the earlier 6M estimate, so Groq's current free-tier limits must be checked before the pilot, and the number of histories is set to fit them.

5. EXPECTED OUTCOMES AND THREATS TO VALIDITY

Threat: limited architectural novelty. The design resembles event sourcing combined with retrieval, as acknowledged in Section 2. The contribution is the measurement, the refresh trigger and the prediction, and the closest systems are run as baselines.

Threat: an unsurprising result. The expected curve may appear unsurprising, and raw-only baselines are flat by construction. The fixed-content arm, the accuracy-per-token frontier, reporting where rewriting *wins*, and explaining the IVA 2026 counter-result [31] are what make it informative.

Threat: storage cost. Reported as tokens and bytes per user per month; BM25S keeps exact search cheap.

Threat: privacy. Crypto-shredding and re-rendering are evaluated for leftover traces using MemCut's protocol [43].

Threat: concurrent work. The REALM, All-Mem, JustMem, Decision Memory and TierMem groups could publish a drift study first; a preprint will be posted as soon as H1 and H2 are measured.

Limitations. Two model sizes from one family, LLM-judged answers and English-dominant benchmarks. All arXiv references were matched to the arXiv API by title and author, but the check is at abstract level; the full texts of [9], [36], [37], [5] and [13], and the IVA result [31], REALM [29], All-Mem [20] and Memora [16], must be read before submission.

6. WORK PLAN

Table 4 gives the plan.

TABLE 4 – NINE-WEEK PLAN TO ACL ROLLING REVIEW

Dates (2026–27)	Work
4–7 Nov	Kill-switch pilot: 30 LoCoMo questions, fixed arm, $k = 0$–8, raw BM25S; check Groq limits
4–9 Nov	Read [29], [20], [16], [5], [6], [13] in full; revise gap; show guide the scope
10–16 Nov	Harness on Memora and LongMemEval; evidence-age tags
17–23 Nov	Main fixed-arm runs (H1 vs H1′); power check; fit $\hat{\eta}$
24–30 Nov	Streaming arm on long histories (H2); preprint if H1 holds
1–7 Dec	Staleness detector, oracle labels, precision and recall
8–14 Dec	Detector-triggered refresh (H3); 70B writer subset
15–28 Dec	Paper writing; router or Tanglish only if time allows
29 Dec–4 Jan	Guide review; ARR submission (4 Jan 2027 AoE)

7. CONCLUSION

Rewriting a second-brain memory from its last rewrite is a drift problem familiar from video coding. This work isolates pure rewrite damage with a fixed-content arm, estimates how fast facts survive each rewrite, and tests whether a raw-anchored memory with a contradiction-and-omission detector beats rewriting in place on the accuracy-per-token frontier. The architecture is not new; the measurement, the survival-rate estimate and the detector are. The first milestone, the fixed-content arm, decides whether the rest is worth building.

REFERENCES

[1] C. Packer *et al.*, "MemGPT: Towards LLMs as Operating Systems," arXiv:2310.08560, 2023.

[2] P. Chhikara *et al.*, "Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory," arXiv:2504.19413, 2025.

[3] W. Xu *et al.*, "A-MEM: Agentic Memory for LLM Agents," arXiv:2502.12110, 2025.

[4] P. Lewis *et al.*, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," arXiv:2005.11401, 2020.

[5] D. Zhang *et al.*, "Useful Memories Become Faulty When Continuously Updated by LLMs," arXiv:2605.12978, 2026.

[6] S. Zerhoudi, J. Mitrovic and M. Granitzer, "The Compaction Cliff in Long-Running AI Agent Memory," arXiv:2608.22752, 2026.

[7] A. Kwon, "Reclaim Evaluation: A Lossy Memory Is Worse Than an Empty One," arXiv:2606.25449, 2026.

[8] T. Wiegand, G. J. Sullivan, G. Bjøntegaard and A. Luthra, "Overview of the H.264/AVC video coding standard," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 13, no. 7, pp. 560–576, 2003.

[9] V. Srinivasan, "Stateless Decision Memory for Enterprise AI Agents," arXiv:2604.20158, 2026.

[10] B. Li, "User as Code: Executable Memory for Personalized Agents," arXiv:2606.16707, 2026.

[11] M. Piehl and M. Ye, "MemFit: Efficient Long-Term Agentic Memory," arXiv:2610.00872, 2026.

[12] R. C. Malo and T. Qiu, "PROJECTMEM: A Local-First, Event-Sourced Memory and Judgment Layer for AI Coding Agents," arXiv:2606.12329, 2026.

[13] A. G. Colaco and N. Lahjouji, "What to Keep, What to Forget: A Rate–Distortion View of Memory Compaction in LLMs and Agents," arXiv:2607.08032, 2026.

[14] V. Dongre *et al.*, "Drift no more? Context equilibria in multi-turn LLM interactions," arXiv:2510.07777, 2025.

[15] A. Maharana *et al.*, "Evaluating Very Long-Term Conversational Memory of LLM Agents," arXiv:2402.17753, 2024.

[16] M. N. Uddin *et al.*, "From recall to forgetting: Benchmarking long-term memory for personalized agents," in *Findings of ACL*, 2026, arXiv:2604.20006.

[17] D. Wu *et al.*, "LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory," arXiv:2410.10813, 2024.

[18] K. Li *et al.*, "TiMem: Temporal-Hierarchical Memory Consolidation for Long-Horizon Conversational Agents," arXiv:2601.02845, 2026.

[19] J. Fang *et al.*, "LightMem: Lightweight and Efficient Memory-Augmented Generation," arXiv:2510.18866, 2025.

[20] C. Lv *et al.*, "All-Mem: Agentic lifelong memory via dynamic topology evolution," arXiv:2603.19595, 2026.

[21] J. Sun *et al.*, "TrajWiki: Source-Grounded Memory Trajectories for Long-Horizon Dialogue Agents," arXiv:2608.00967, 2026.

[22] P. Banerjee *et al.*, "APEX-MEM: Agentic semi-structured memory with temporal reasoning," in *Proc. ACL*, 2026. [Online]. Available: https://aclanthology.org/2026.acl-long.749/

[23] Z. Shen *et al.*, "AnchorMem: Anchored facts with associative contexts," in *Findings of ACL*, 2026. [Online]. Available: https://aclanthology.org/2026.findings-acl.1736/

[24] Y. Su *et al.*, "ChronoMem: Version Control and Semantic Rollback for Large Language Model Agent Memory," arXiv:2607.27773, 2026.

[25] S. Yu *et al.*, "Shepherd: Enabling Programmable Meta-Agents via Reversible Agentic Execution Traces," arXiv:2605.10913, 2026.

[26] H. Tan *et al.*, "DELTAMEM: Incremental Experience Memory for LLM Agents via Residual Trees," arXiv:2606.03083, 2026.

[27] CortexDB, "Event sourcing," documentation. [Online]. Available: https://cortexdb.ai/docs/concepts/event-sourcing

[28] C. Lam *et al.*, "Governing Evolving Memory in LLM Agents: Risks, Mechanisms, and the Stability and Safety Governed Memory (SSGM) Framework," arXiv:2603.11768, 2026.

[29] Y. Song *et al.*, "Retrieval-driven memory reconsolidation for long-term LLM agents," arXiv:2609.16053, 2026.

[30] D. Keresteciogl u *et al.*, "Human-inspired memory architecture for LLM agents," arXiv:2605.08538, 2026.

[31] E. Butts and S. Daher, "Consistent conversational state for virtual agents: Slot-based memory for accurate fact retrieval," in *Proc. ACM IVA*, 2026, doi: 10.1145/3806774.3832787.

[32] P. Rasmussen *et al.*, "Zep: A Temporal Knowledge Graph Architecture for Agent Memory," arXiv:2501.13956, 2025.

[33] Z. Tang *et al.*, "Mnemis: Dual-Route Retrieval on Hierarchical Graphs for Long-Term LLM Memory," arXiv:2602.15313, 2026.

[34] M. McKee, "SelRoute: Query-Type-Aware Routing for Long-Term Conversational Memory Retrieval," arXiv:2604.02431, 2026.

[35] H. Zhang *et al.*, "Learning Query-Aware Budget-Tier Routing for Runtime Agent Memory," arXiv:2602.06025, 2026.

[36] G. Chen *et al.*, "JustMem: Just-Enough Memory Access for Long-Term Conversations," arXiv:2609.19877, 2026.

[37] Q. Zhu *et al.*, "From Lossy to Verified: A Provenance-Aware Tiered Memory for Agents," arXiv:2602.17913, 2026.

[38] Y. Si *et al.*, "RENDER: Controlling Reader-Facing Evidence in LLM Memory Evaluation," arXiv:2608.23568, 2026.

[39] X. H. Lù, "BM25S: Orders of magnitude faster lexical search via eager sparse scoring," arXiv:2407.03618, 2024.

[40] M. Wu and P. Zhu, "Agent Zero Memory: Provenance-Aware Long-Term Memory for LLM Agents," arXiv:2608.29606, 2026.

[41] J. Xu *et al.*, "Memory Provenance Laundering in LLM Agents: A Non-Amplification Firewall for Persistent Memory," arXiv:2607.29167, 2026.

[42] X. Zhou *et al.*, "MemTrust: A Zero-Trust Architecture for Unified AI Memory System," arXiv:2601.07004, 2026.

[43] C. Zhang *et al.*, "MemCut: Every Secret Leaves a Trail in the Long-Term Memory of LLM Agents," arXiv:2610.21405, 2026.

[44] Z. Wang, "TOKI: A Bitemporal Operator Algebra for Contradiction Resolution in LLM-Agent Persistent Memory," arXiv:2606.06240, 2026.

[45] W. Huang *et al.*, "AMA: Adaptive Memory via Multi-Agent Collaboration," arXiv:2601.20352, 2026.

[46] Y. Polyanskiy and Y. Wu, "Strong data-processing inequalities for channels and Bayesian networks," in *Convexity and Concentration*, Springer, 2017, pp. 211–249, arXiv:1508.06025.

[47] Y. Madhani *et al.*, "Aksharantar: Open Indic-language transliteration datasets and models for the next billion users," in *Findings of EMNLP*, 2023, arXiv:2205.03018.

[48] S. D. Gawande and M. Singh, "IndicTalk: A Large-Scale Persona-Based Multilingual Conversational Corpus for Indic Languages," arXiv:2607.23242, 2026.

[49] A. Grattafiori *et al.*, "The Llama 3 herd of models," arXiv:2407.21783, 2024.